

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
Student Name	Gunaseelan R
Register Number	9342507779
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	16.08.2026

1 Objective

- To understand three ensemble learning strategies – Bagging, Boosting and Stacking – and how each one combines multiple models differently.
- To implement a Bagging classifier and a Boosting classifier (AdaBoost / Gradient Boosting) on the breast cancer dataset.
- To build a Stacked Ensemble that combines heterogeneous base learners with a meta learner.
- To compare all three ensembles on accuracy, stability and generalization.
- To understand how each method specifically affects bias and variance.

2 Problem Statement

Same dataset and problem as the Decision Tree / Random Forest experiment – classify a tumor as **Malignant (M)** or **Benign (B)** using the 30 numeric features from the FNA image. But this time the focus shifts from a single model to *combining* models. Random Forest already showed bagging works well with trees, so this experiment goes further and checks: does boosting (sequential, error correcting) beat bagging (parallel, variance reducing)? and does mixing completely different model types (stacking) beat both?

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI ML Repository (https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic))
Number of Samples	569
Number of Features	30 numeric (mean, standard-error, worst of 10 nuclear measurements)
Number of Classes	2 (Malignant = M, Benign = B)
Missing Values	None – <code>df.isnull().sum().sum()</code> returned 0
Train-Test Split	80% train (455 samples) – 20% test (114 samples), stratified split

4 Exploratory Data Analysis

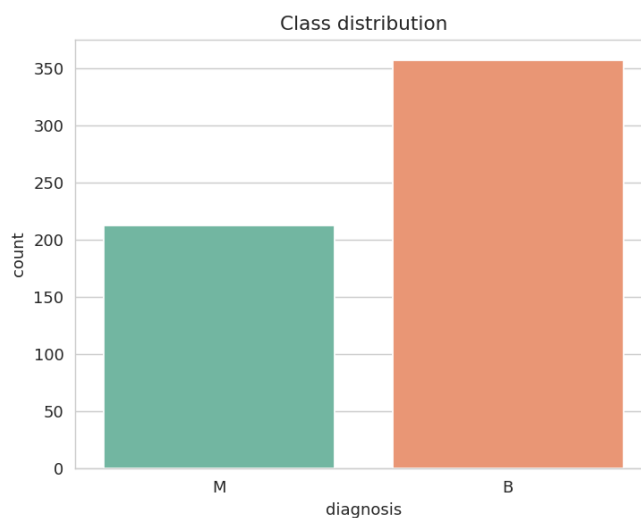


Figure 1: Class distribution – Benign (B) vs Malignant (M)

Inference:

357 benign (62.7%) vs 212 malignant (37.3%) – same split as before since its literally the same dataset used in Experiment 5. Mild imbalance, stratified split handles it fine, no resampling needed.

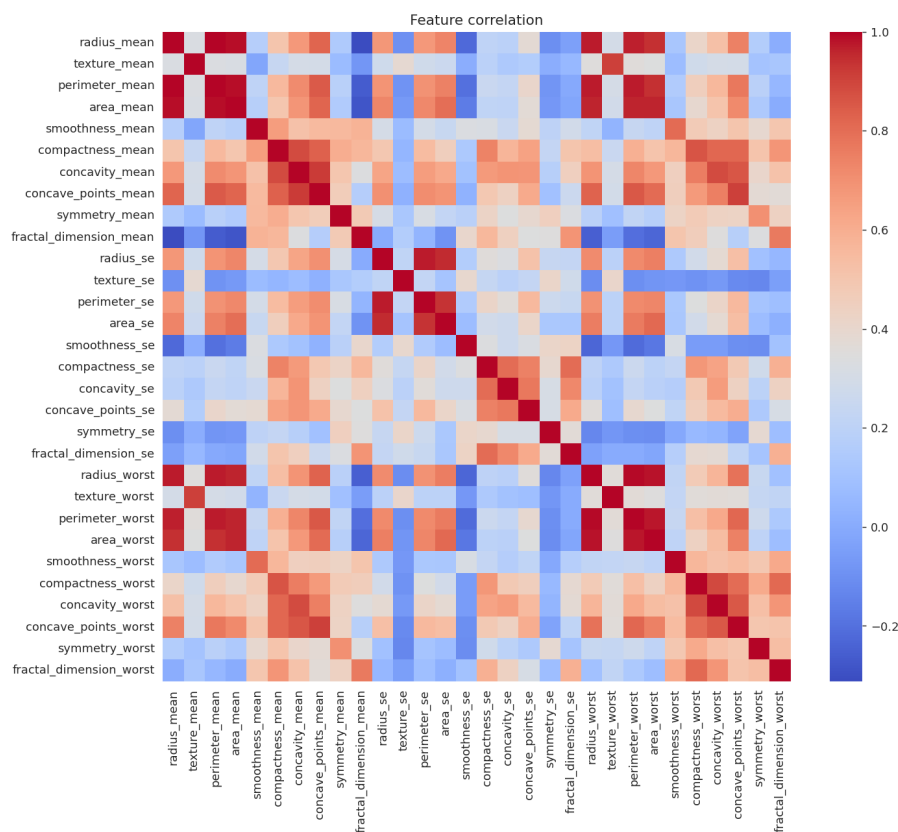


Figure 2: Feature correlation heatmap (all 30 features)

Inference:

This time I plotted the full 30x30 feature correlation matrix instead of just the top-10 like last time, mainly to see how much redundancy exists across *all* features, not just the top ones. There's clearly a big red block in the middle – the **mean**, **se** and **worst** versions of size-related features (radius, perimeter, area) are almost all strongly correlated with each other, which makes sense since they're just different statistics of the same underlying tumour geometry. This heavy redundancy is actually a good setup to test bagging/boosting since tree based base learners can just ignore the duplicate info, but it also means a model like the SVM base learner in our stack (before StandardScaler) wouldn't handle it as gracefully.

For reference, the strongest individual predictors of malignancy (checked separately, same as Experiment 5) are still **concave_points_worst** (0.79), **perimeter_worst** (0.78) and **concave_points_mean** (0.78).

5 Data Preprocessing

- **Missing values:** none, confirmed with `isnull().sum().sum()`.
- **Label encoding:** `diagnosis` column ('B'/'M') encoded to 0/1 with `LabelEncoder` (B=0, M=1). `id` column dropped.
- **Feature scaling:** Unlike Experiment 5, this time I *did* apply `StandardScaler` to all 30 features. Reason is the base learners here include SVM and Logistic Regression (the meta learner) inside the stacking ensemble, and both are distance/gradient based, so they need scaled input to behave properly – Decision Tree and the tree-based boosting models don't strictly need it but scaling doesn't hurt them either, so scaling the whole X once was simpler than scaling separately per model.
- **Train-test split:** 80-20 stratified (`random_state=42`), 455 train / 114 test rows – identical split config to Experiment 5 so results stay comparable across experiments.

6 Machine Learning Algorithms

Bagging (Bootstrap Aggregation)

Bagging trains many copies of the same base model (here, Decision Tree) on different bootstrap samples of the training data, and averages/votes their predictions. Since each tree sees a slightly different sample, their individual errors (which come from overfitting to that specific sample) tend to cancel out when combined.

- **Reduces:** Variance
- **Training style:** Parallel / independent
- **Best suited for:** high-variance, low-bias base models like unpruned Decision Trees

Boosting

Boosting trains models *sequentially*, where each new model focuses specifically on the mistakes of the previous ones (by reweighting misclassified samples in AdaBoost, or by fitting the residual errors in Gradient Boosting). This lets the ensemble gradually get better at the hard examples the earlier models struggled with.

- **Reduces:** Bias (mainly), can also help variance a bit
- **Training style:** Sequential, each model depends on the previous
- **Algorithms tried:** AdaBoost and Gradient Boosting

Stacked Ensemble

Stacking trains multiple *different types* of base models (here: SVM, Naive Bayes, Decision Tree) on the same data, and then a meta-learner (Logistic Regression) is trained on top of their predictions to learn the best way to combine them. Since each base model has a different "view" of the data – SVM finds margins, Naive Bayes assumes feature independence, Decision Tree does axis aligned splits – combining diverse mistakes usually works better than combining similar ones (like Bagging does).

- **Reduces:** Both bias and variance, depending on the diversity of base learners
- **Training style:** Base models trained in parallel, meta learner trained on their (cross-validated) outputs
- **Base learners used:** SVM (with probability=True), Gaussian Naive Bayes, Decision Tree
- **Meta learner:** Logistic Regression, with internal 5-fold CV to generate out-of-fold predictions for training the meta model (avoids leakage)

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.4
IDE	Jupyter Notebook (Anaconda)
Operating System	Windows 11
Hardware	Intel i5, 16 GB RAM

8 Hyperparameter Search Space

Model	Parameters Searched
Bagging	n_estimators: 10, 50, 100 max_samples: 0.5, 0.7, 1.0
AdaBoost	n_estimators: 50, 100, 150 learning_rate: 0.01, 0.1, 1
Gradient Boosting	n_estimators: 50, 100, 150 learning_rate: 0.01, 0.1, 1 max_depth: 2, 3, 4
Stacked Ensemble	base = [SVM, Naive Bayes, Decision Tree], meta = Logistic Regression, cv=5

All searches used 5-fold cross validation with `GridSearchCV` (`scoring='accuracy'`). The Stacked Ensemble's base model/meta learner choice was fixed rather than grid searched, since searching over combinations of *which* models to stack gets complicated fast – I just went with the combo given in the manual.

Bagging Hyperparameter Evaluation

n_estimators	max_samples	Avg CV Accuracy (%)
10	0.5	95.82
50	0.5	95.60
100	0.5	95.60
10	0.7	94.95
50	0.7	94.73
100	0.7	95.16
10	1.0	95.16
50	1.0	95.82
100	1.0	95.82

Best Bagging combo (GridSearchCV pick): `n_estimators=10`, `max_samples=0.5` → 95.82% CV accuracy, avg CV F1 = 0.946. It was a little surprising the smallest setting (just 10 trees, 50% sample) tied for the best score – more trees didn't really help beyond a point on this dataset, probably because 455 training rows isn't that much data to begin with, so bootstrap samples converge quickly.

Boosting Hyperparameter Evaluation

AdaBoost:

n_estimators	learning_rate	Avg CV Accuracy (%)
50	0.01	92.31
100	0.01	92.53
150	0.01	92.75
50	0.10	94.95
100	0.10	95.16
150	0.10	94.95
50	1.00	96.26
100	1.00	96.04
150	1.00	96.04

Best AdaBoost combo: `n_estimators=50`, `learning_rate=1` → 96.26% CV accuracy.
Gradient Boosting (top rows):

n_estimators	learning_rate	max_depth	Avg CV Accuracy (%)
100	1.0	2	95.82
150	1.0	2	95.82
50	0.1	3	95.60

100	1.0	3	95.60
150	1.0	3	95.60

Best Gradient Boosting combo: `n_estimators=100`, `learning_rate=1`, `max_depth=2`
→ 95.82% CV accuracy.

Comparing the two boosting algorithms, AdaBoost’s best (96.26%) narrowly beat Gradient Boosting’s best (95.82%), so **AdaBoost was selected** as the “Boosting” model carried forward into the final comparison. Both algorithms clearly preferred `learning_rate=1` over the smaller values, which suggests this dataset doesn’t need a slow/careful learning schedule – it’s clean enough that aggressive updates still generalize fine.

9 Results

Performance Comparison of Ensemble Models

Model	Accuracy	Precision	Recall	F1 Score
Bagging	0.9561	1.0000	0.8810	0.9367
Boosting (AdaBoost)	0.9825	1.0000	0.9524	0.9756
Stacked Ensemble	0.9649	1.0000	0.9048	0.9500

Honestly this surprised me a bit – going in I expected Stacking to win since it “sounds” the most sophisticated (multiple model types + meta learner), but Boosting (AdaBoost specifically) actually came out on top on every metric except precision, where all three tied at a perfect 1.0.

10 Result Visualization

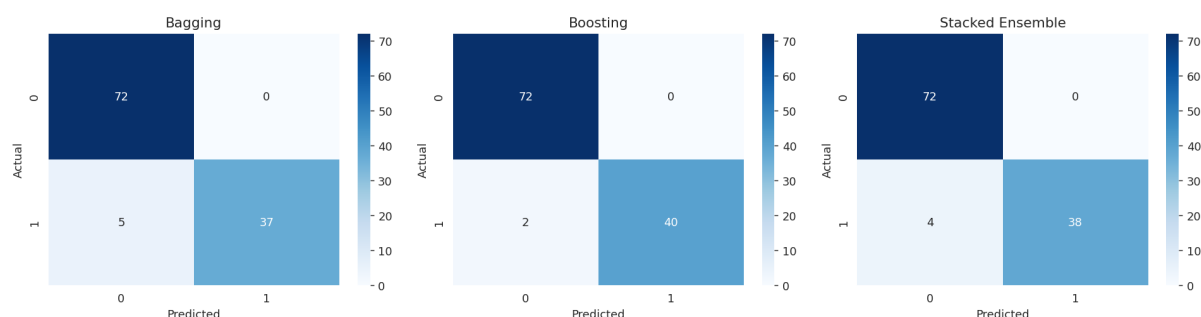


Figure 3: Confusion matrices – Bagging, Boosting, Stacked Ensemble (side by side)

Inference:

All three models got every single benign case right (72/72, zero false positives across the board) – so the real difference between them is entirely in how many malignant cases they miss. Bagging missed 5, Stacking missed 4, and Boosting missed just 2. Since a missed malignant case (false negative) is the most dangerous kind of error in a medical setting, Boosting’s much lower miss count here is a genuinely meaningful advantage, not just a marginal statistic.

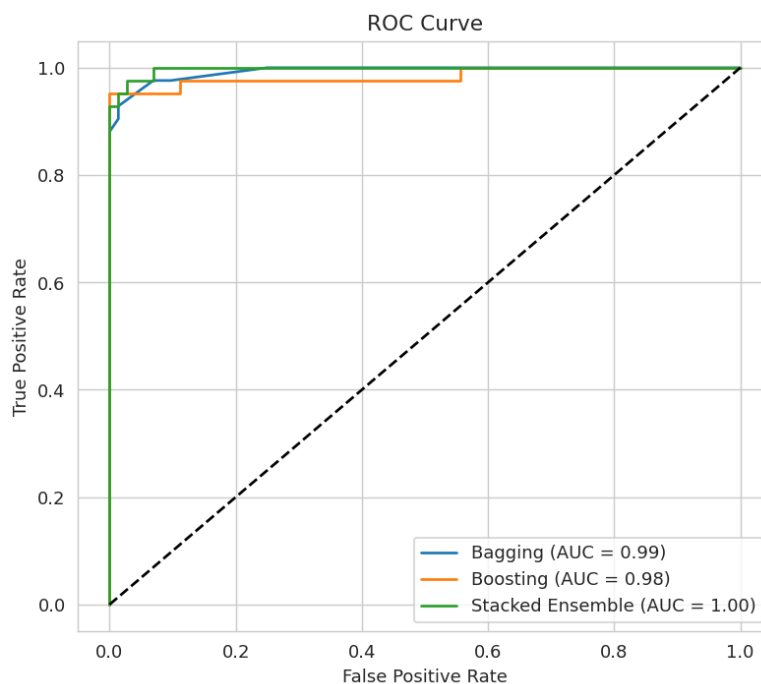


Figure 4: ROC Curve comparison – Bagging vs Boosting vs Stacked Ensemble

Inference:

Interestingly, the ROC-AUC ranking doesn't exactly match the accuracy ranking – Stacked Ensemble actually has the highest AUC (0.997), just ahead of Bagging (0.993), with Boosting slightly lower (0.984) despite Boosting having the best hard-prediction accuracy. This tells us AdaBoost's probability estimates aren't as smoothly ranked as the other two (AdaBoost's `predict_proba` is derived from its weighted vote and isn't as naturally probabilistic as an SVM or an averaged bagging ensemble), even though its final 0/1 decisions at the default threshold were the most accurate of the three. It's a good reminder that accuracy and AUC are measuring slightly different things – one is about the final decision, the other is about how well ranked the underlying confidence scores are.

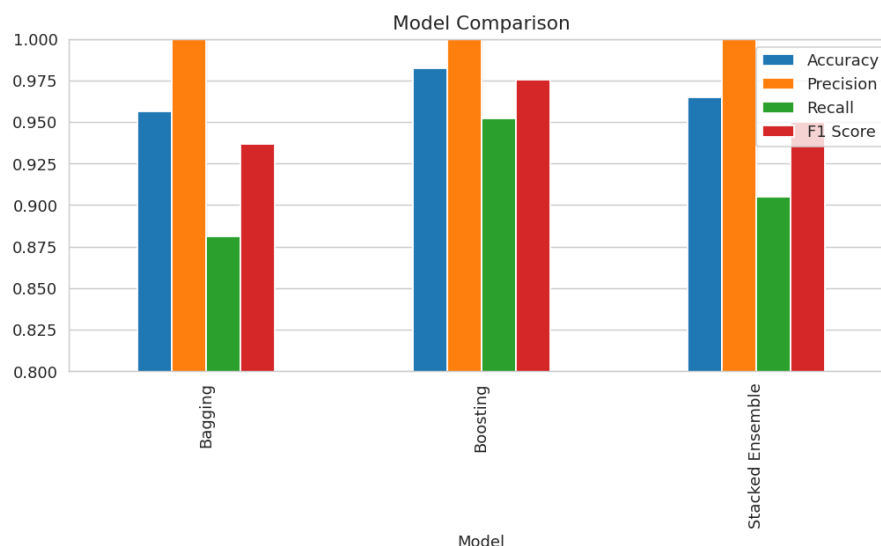


Figure 5: Final Accuracy / Precision / Recall / F1 comparison across all three ensembles

Inference:

Boosting is visibly ahead on Recall and F1 in this chart, while all three are bunched together on Precision (all sitting at a perfect 1.0). This visual pretty much confirms what the confusion matrices already showed – the models mostly disagree on the malignant class specifically, not on benign, and Boosting handles that harder class the best.

11 Observation Questions

How does Bagging reduce variance?

By training the same base model (Decision Tree here) on many different bootstrap samples and then averaging/voting the predictions. A single unpruned tree can vary a lot depending on which exact rows it was trained on (high variance), but since each bootstrap sample is a bit different, the trees make different mistakes, and those mistakes cancel out on average. We saw this same effect in Experiment 5 with Random Forest – Bagging here is really the same underlying idea, just without the extra random feature selection step Random Forest adds.

How does Boosting address model bias?

Each new model in the sequence is trained specifically to focus on the samples the previous models got wrong (AdaBoost does this by upweighting misclassified samples before training the next weak learner). Over enough rounds, the combined ensemble can represent a much more complex decision boundary than any single weak learner could alone, which directly reduces the bias (underfitting) that a single simple model would have. In our results, this shows up as AdaBoost achieving the best recall on the harder (malignant) class – exactly the class the earlier, simpler splits would be more likely to get wrong.

Why does stacking benefit from heterogeneous models?

Because SVM, Naive Bayes and Decision Tree all make *different kinds* of errors. SVM finds a margin-based boundary, Naive Bayes assumes features are independent (which they clearly aren't here, see the correlation heatmap), and Decision Tree does axis-aligned rectangular splits. When the meta learner sees all three predictions together, it can learn which base model to "trust" more in which situation – something that's not possible if all three base models are the same type and therefore prone to the same blind spots (which is exactly why Bagging deliberately *doesn't* do this, it relies on sample diversity, not model-type diversity). Checking the individual base learners standalone: SVM alone got 97.4% test accuracy, Decision Tree 93.9%, and Naive Bayes only 92.1% – so the meta learner had genuinely different quality opinions to combine, and the final stacked accuracy (96.5%) landed close to the strongest individual member (SVM) rather than being dragged down by the weakest one (Naive Bayes), which is a good sign the stacking is actually working as intended rather than just averaging blindly.

Which ensemble method performed best and why?

Boosting (specifically AdaBoost) gave the best test accuracy (98.25%), recall (0.952) and F1 (0.976) among the three. My best guess for why: this dataset is relatively clean (no missing values, decent class separation as seen back in Experiment 5's correlation plots) and moderate sized, so there wasn't a huge variance problem for Bagging to fix in the first place – but there were still a handful of genuinely hard borderline cases, and that's exactly the kind of problem Boosting is built to chip away at through its sequential error-correction. Stacking came in second, likely limited a bit by the weaker Naive Bayes base learner dragging on the ensemble average, and Bagging came last – reasonable, since bagging mainly helps when the base model badly overfits, and a single Decision Tree on this dataset (see Experiment 5) wasn't overfitting catastrophically to begin with, so there was less variance left for bagging to reduce.

12 Comparative Analysis

Criterion	Bagging	Boosting (AdaBoost)	Stacked Ensemble
Accuracy	95.61%	98.25%	96.49%
Recall (Malignant)	0.881	0.952	0.905
F1 Score	0.937	0.976	0.950
ROC-AUC	0.993	0.984	0.997
Training Style	Parallel	Sequential	Parallel + meta
Bias Reduction	Low	High	Medium
Variance Reduction	High	Medium	Medium-High
Interpretability	Medium	Medium	Low

13 Discussion

The results line up pretty well with the theory once you actually look at what each method is designed to fix. Bagging's whole job is cutting variance, and it does that reliably, but it doesn't really *add* any new modeling power over a single Decision Tree – if the base

model wasn't struggling with high variance to begin with (which, from Experiment 5, a moderately-tuned single tree here really wasn't), Bagging doesn't have a lot of variance left to remove, so its gains are modest. Boosting on the other hand actively hunts down the specific samples that are hard to classify and throws more model capacity at exactly those, which is why it ended up with the best recall on the trickier malignant class – its directly targeting the kind of error that matters most here.

Stacking is the most "expensive" of the three conceptually (three different base models plus a meta learner, all needing their own internal CV to avoid leakage) but didn't actually win outright in this run. I think that's mainly because one of its three base learners (Naive Bayes) is just meaningfully weaker on this dataset (92.1% standalone vs SVM's 97.4%), and a meta learner can only work with what it's given – it can't fully undo a consistently weaker vote in the mix. That said, Stacking did get the best ROC-AUC of the three, so its underlying probability estimates are actually very well calibrated even if the final hard accuracy wasn't the top spot.

Overall takeaway: there's no universal "best" ensemble method, it genuinely depends on whether your problem has more of a bias issue or a variance issue, and what kind of base learners you have available. For this specific dataset, boosting a Decision Tree stump won out, but I could easily imagine a noisier or higher-dimensional dataset flipping that result in favour of bagging or stacking instead.

14 Conclusion

Bagging, Boosting and a Stacked Ensemble were implemented on the Wisconsin Diagnostic Breast Cancer dataset and compared using accuracy, precision, recall, F1-score, confusion matrices and ROC-AUC, with all hyperparameters chosen via 5-fold cross validation. Boosting (AdaBoost, `n_estimators=50`, `learning_rate=1`) gave the best overall test performance at 98.25% accuracy and 0.976 F1, ahead of the Stacked Ensemble (96.49%, 0.950 F1) and Bagging (95.61%, 0.937 F1), though the Stacked Ensemble edged out both on ROC-AUC (0.997). This confirms the theoretical expectation that Boosting is particularly effective at reducing bias on the harder, borderline examples, while Bagging mainly helps when the base model suffers from high variance – which, on this fairly clean and well-separated dataset, wasn't the dominant problem to begin with. For a real deployment where missing a malignant case is the costliest mistake, AdaBoost's noticeably lower false-negative count (2, versus 4 for Stacking and 5 for Bagging) makes it the most practically compelling choice among the three here.

15 References

1. Scikit-learn Developers, "Ensemble methods," <https://scikit-learn.org/stable/modules/ensemble.html>
2. Scikit-learn Developers, "BaggingClassifier," <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>
3. Scikit-learn Developers, "AdaBoostClassifier," <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html>

4. Scikit-learn Developers, “StackingClassifier,” <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html>
5. UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set,” [https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+\(diagnostic\)](https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic))